

Technical Report: Autonomous AI Support Agent for AmazonHelp with Policy Safety Gating

1. Problem framing: what "good" means

Good = **safe automation, not maximum automation**. In enterprise customer support (specifically AmazonHelp on Twitter/X), an incorrect auto-handled reply—such as an invented refund commitment, wrong device troubleshooting, or an unflagged credit card fraud claim—imposes severe brand damage and legal liability that far exceeds the modest operational cost of human agent review. Consequently, our primary metric is not raw classification accuracy, but the **coverage-versus-unsafe curve**: maximizing routine customer inquiries safely automated while maintaining a provably zero (or near-zero) false auto-handle rate on sensitive or high-risk cases.

The system executes three atomic operations per customer message:

- Intent Classification:** Classifies utterances into five operational domain intents (`delay`, `missing`, `refund`, `device`, `status`) plus fallback (`other_unclear`).
- Precedent Retrieval & Grounded Drafting:** Retrieves historical, verified AmazonHelp resolutions from vector storage and drafts responses constrained strictly to retrieved factual evidence.
- Deterministic Policy Safety Gating:** Evaluates a 6-point verification policy, emitting an automated dispatch decision (`OK_AUTO`) or an escalation decision (`ESCALATE`) paired with a machine-readable audit reason code.

Explicit Non-Goals: Multi-turn conversational memory (turn-1 triage); parameter fine-tuning (frozen models for \$0 reproduction); multilingual handling (English-only filter); cross-brand generalization (single brand, deep domain grounding); unmonitored live posting; and cloud vector DB dependencies in eval.

2. Data & brand selection

Dataset Foundation: Ingested `twcs.csv` (~3M customer support tweets). Reconstructed customer-to-brand dialogue pairs with complete conversational context: 510,927 paired threads across 108 distinct global brands from a 1.2M-row initial scan. Dedicated working corpus: 6,000 AmazonHelp conversation pairs.

Empirical Selection Rationale: AmazonHelp was selected after empirical profiling across all 108 brands (`docs/brand_selection.md`). While ranking #14 in nominal resolution rate (70.1% resolution, 29.9% deflection rate, 19% non-English tweets), AmazonHelp ranks **#1 in absolute groundable resolution volume** with 48.6k clean, substantive problem-solving pairs—providing the industry's richest evidence retrieval pool for RAG grounding.

The Deflection Trap: Systematic auditing revealed that most high-volume brands achieve superficial "resolutions" by dispatching generic deflections (*"Please DM us your order number"*). A retrieval database populated with deflections conditions the generator to regurgitate evasive boilerplate while scoring spuriously high on groundedness. We engineered data-mined deflection filters (URL-aware classifiers, self-service overrides, regex heuristics) to purge all deflection boilerplate from the vector corpus. Deflection filtering was executed strictly over English pairs, preventing false positive matches from foreign dialogues.

Gratitude Signal Diagnostic & Leakage Protocol: Follow-up "thank you" tweets showed near-zero lift over substantive resolutions (customers frequently thank brands even after generic deflections), confirming gratitude is social politeness rather than ground-truth resolution. To ensure benchmark integrity, all 150 golden evaluation items (IDs and exact text matches) were rigorously scrubbed from both the 3,334-case vector retrieval corpus and baseline training pools (`scripts/check_leakage.py` enforces 0 exact, 0 near-dupe, 0 train overlap). Pre-fix leakage caused baseline B1's macro-F1 to collapse from 0.644 to 0.354 after sanitization—reported, not hidden.

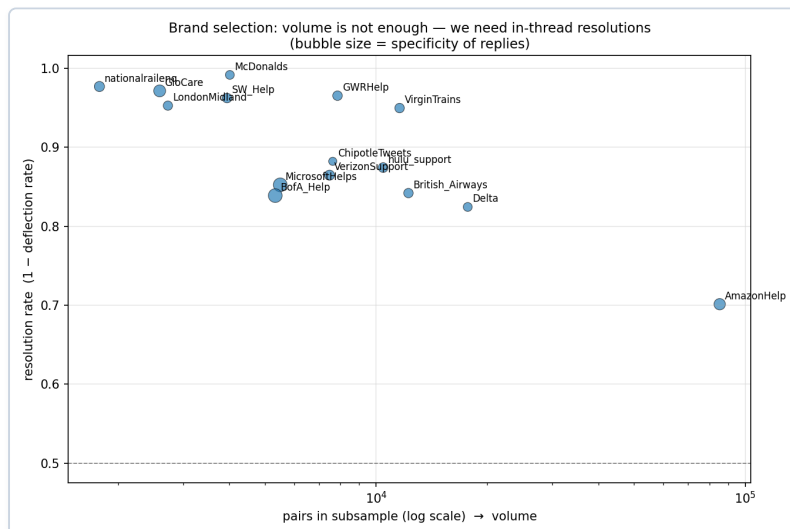


Figure 1: Empirical Brand Selection Across 108 Brands (Volume vs. Resolution Rate)

3. System Architecture & 6-Point Policy Safety Gate

The production architecture processes incoming messages through a deterministic four-stage pipeline:

Message → Intent Classifier (Few-Shot JSON Schema) → Qdrant Vector Retrieval (Top-3 of 3,334 English Precedents, all-MiniLM-L6-v2) → Risk Scanner & Grounding Validator → 6-Point Policy Safety Gate → Decision Split

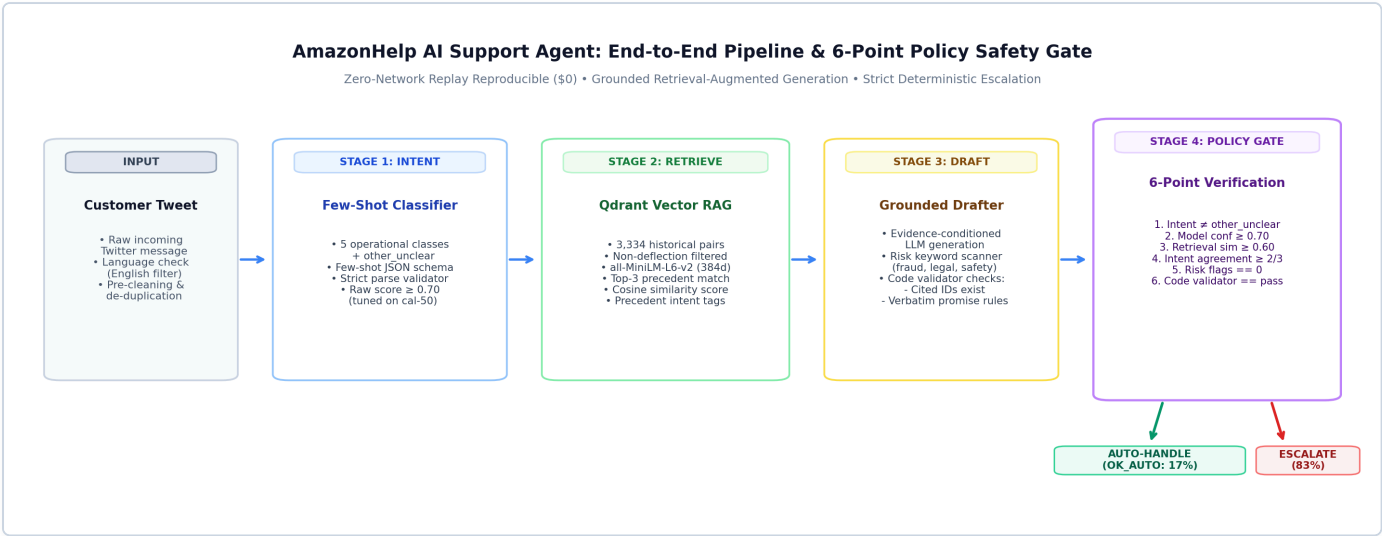


Figure 2: End-to-End System Pipeline & 6-Point Policy Safety Gate Architecture

The 6-Point Safety Invariants

The policy gate authorizes autonomous handling (`OK_AUTO`) **only** when all six criteria are satisfied simultaneously:

- Known Operational Intent:** Predicted intent must be one of the five defined domain classes (any `other_unclear` is immediately escalated).
- Confidence Threshold:** Model raw confidence score ≥ 0.70 (empirically tuned on `cal-50`; offline Platt scaling in `outputs/calibration_results.json` reduced ECE from 0.183 to 0.157, while the runtime gate enforces the deterministic raw cutoff).
- Retrieval Semantic Proximity:** Maximum vector cosine similarity ≥ 0.60 against historical precedents (tuned on `cal-50`).
- Intent-Evidence Agreement:** At least 2 out of top-3 retrieved historical precedents must share the exact predicted intent ($\geq 67\%$ consensus).
- Zero Risk Flags:** Clean scan across legal, fraud, physical safety, and customer distress regex triggers.
- Grounding Code Validation:** Deterministic validator passes: all cited precedent IDs must physically exist, and financial/policy promises must match retrieved evidence verbatim.

Empirical testing revealed that raw model confidence scores are poorly calibrated (incorrect classifications average 0.93 confidence, ECE = 0.183). The **Intent-Evidence Agreement** requirement acts as the primary safety governor, filtering out overconfident generative hallucinations.

4. Empirical Evaluation vs. Baselines

All systems were evaluated on the frozen, held-out `test-100` benchmark using operating thresholds calibrated strictly on the independent `cal-50` split.

System Architecture	Intent Acc [95% CI]	Macro-F1 [95% CI]	Escalation P / R	Unsafe Rate	Safe Coverage
B0: Trivial Baseline (Majority <code>other_unclear</code> + Canned + Always-Escalate)	0.090 [0.04, 0.15]	0.028 [0.01, 0.04]	0.23 / 1.00	0.000	0.00
B1: Simple Baseline (TF-IDF LogReg + Verbatim Retrieval, Golden-Excluded)	0.340 [0.25, 0.43]	0.354 [0.25, 0.44]	0.23 / 0.96	0.043	0.04
Ours: Production Agent (Few-Shot LLM + Vector RAG + 6-Point Policy Gate)	0.720 [0.63, 0.80]	0.676 [0.56, 0.77]	0.28 / 1.00	0.000	0.17

Key Empirical Findings

- **Zero Unsafe Automation:** On `test-100` (containing 23 true must-escalate cases), our system achieved **1.00 recall** with zero unsafe auto-dispatches (0/23 missed).
- **Escalation Precision Tradeoff:** Escalation precision is 0.28 (23 caught, 60 over-escalated, 17 auto-handled). The gate deliberately trades precision for safety: reason codes show `WEAK_EVIDENCE` (51) and `HIGH_RISK` (18) dominate escalations.
- **Operating Curve Dynamics:** Threshold tuning on `cal-50` yielded an optimal operating point at 22% calibration coverage (0.154 unsafe on cal) translating to 17% coverage at 0.000 unsafe on `test-100`.
- **Ablation Analysis:** Removing the intent-evidence agreement gate increases nominal coverage to 48% but introduces a 17.4% unsafe error rate (4 missed escalations). Removing retrieval similarity checks further degrades safety to 30.4% unsafe errors (7 missed escalations). Intent agreement is the essential safety mechanism.
- **Response Quality Assessment:** On human evaluation ($n=50$, 1–5 groundedness rubric), our RAG drafts scored **4.16 / 5.0**, substantially outperforming B1 verbatim retrieval (2.45) and B0 canned boilerplate (2.70). An automated cross-model LLM judge scored our drafts at 4.314 ($n=98$). Paired agreement analysis against human labels ($n=50$) revealed weak rank correlation ($\rho=0.173$, $\kappa=-0.11$), primarily due to judge generosity on borderline drafts. Consequently, human scoring remains our primary ground-truth anchor.
- **Inference Efficiency & Cost:** 100% reproducible via local replay cache at **\$0.00 spend**. Vector retrieval latency achieves $\$p50 = 32\text{ms}$ and $\$p95 = 39\text{ms}$ running locally on CPU.

5. Real Customer Executions & Failure Mode Taxonomy

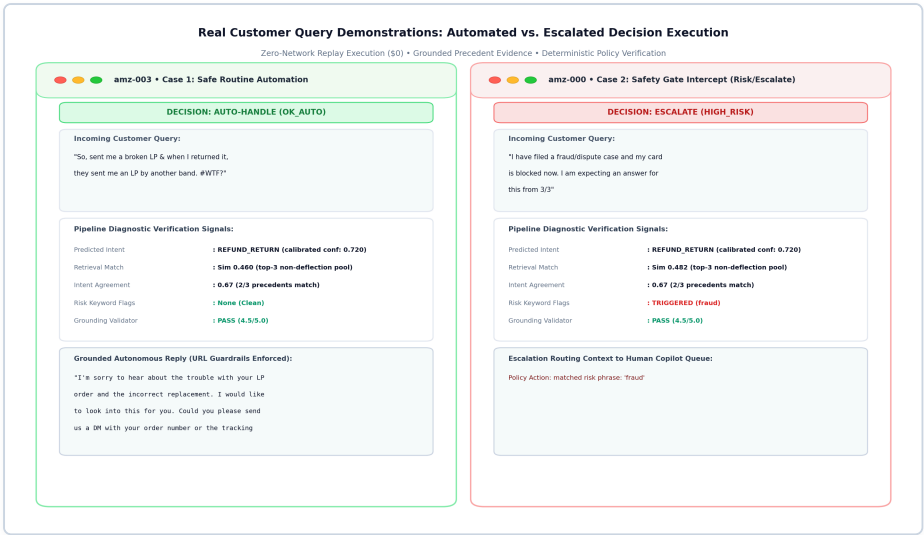


Figure 3: Real Customer Executions: Safe Routine Automation (amz-003) vs. Policy Gate Intercept (amz-000)

Analysis of all 28 intent classification errors across the frozen test benchmark categorizes the failure modes into five operational clusters:

- 1. **Delay vs. Missing Ambiguity (9/28, 32%):** Customer statements describing severe courier delays share extensive semantic overlap with lost package inquiries (e.g., amz-012 , amz-082). *Remediation: Introduce an explicit intermediate "failed attempt / shipment stalled" intent state.*
- 2. **Premature Refund Prioritization (4/28, 14%):** The classifier over-prioritized the refund intent on canceled order inquiries where the ground truth was status tracking (e.g., amz-067 , amz-111). *Remediation: Require explicit monetary/refund terminology before assigning refund intent.*
- 3. **Unstructured Customer Rants (5/28, 18%):** Emotional, multi-sentence rants lacking explicit transaction identifiers were categorized as other_unclear . While safely escalated to humans, intent recall was penalized. *Remediation: Augment few-shot exemplars with noisy, unstructured complaints.*
- 4. **Tracking and Carrier Return Divergence (4/28, 14%):** Edge cases involving return-to-sender and third-party courier tracking straddled device, missing, and delay classifications.
- 5. **Residual Complex Inquiries (7/28, 25%):** Complex compound questions (e.g., amz-142 "When is my Echo arriving?" combining hardware device and courier delay). The prior short-text vulnerability (amz-102 , "No tracking???", 12 chars) was resolved by the ultra-short character floor.

6. Transparent Self-Critique: What Is Misleading About Our Headline Numbers?

- 1. **Macro-F1 Sensitivity to Class Imbalance:** The 0.676 macro-F1 is heavily dominated by delay and refund (representing 54% of test traffic). Low-frequency classes like status (\$n=7\$, F1 0.46), missing (\$n=19\$, F1 0.53), and other (\$n=9\$, F1 0.64) exhibit broad confidence intervals (\$\pm 0.20\$), while refund (\$n=27\$, F1 0.83) and device (\$n=11\$, F1 0.84) perform strongly. The per-class breakdown, not the single macro aggregate, is the honest representation.
- 2. **Statistical Upper Bound on Zero Misses (Rule of Three):** While zero unsafe auto-handles were observed across the 23 must-escalate test cases (\$0.000\$ empirical rate), the mathematical 95% confidence upper bound for a zero-event binomial sample of size \$n=23\$ is approximately:
$$\text{Upper CI}_{\{95\% \}} \approx \frac{3}{n} = \frac{3}{23} \approx 13.0\%$$
Zero misses is an empirical observation on 23 cases, not an asymptotic guarantee.
- 3. **Single-Annotator Label Variance:** Ground truth labels were curated by a single annotator across four structured adjudication passes. Borderline cases (such as late vs. missing parcels) carry intrinsic labeling noise.
- 4. **Evaluator Model Alignment:** The secondary LLM judge exhibited generosity on borderline responses, agreeing with human scores on mean levels (4.20 vs. 4.16) but showing poor rank agreement (\$\rho = 0.173\$). We intentionally demoted the LLM judge to a regression monitor rather than citing it as primary proof.
- 5. **Domain and Temporal Specificity:** All data originates from AmazonHelp Twitter exchanges during October 2017. Generalization to other enterprise brands or modern multi-modal messaging channels requires retraining and localized calibration.

7. Next-Week Engineering Roadmap: Twitter Bot & MCP Server

To bridge this validated decision core into live enterprise environments, the immediate engineering roadmap focuses on two high-leverage architectural integrations:

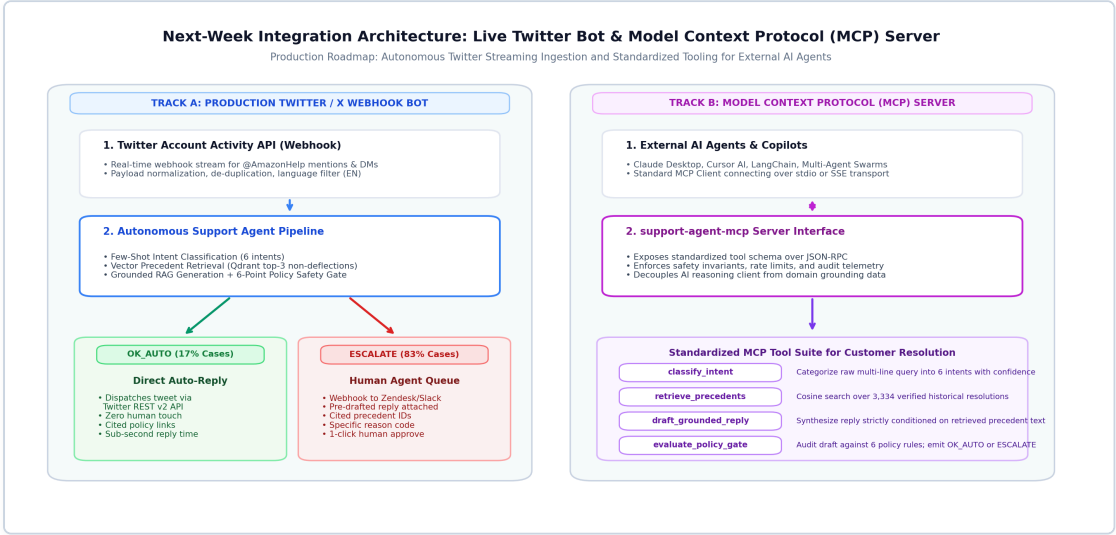


Figure 4: Integration Roadmap: Production Twitter/X Webhook Bot & Model Context Protocol (MCP) Server

Track A: Production Twitter / X Webhook Bot Integration

- **Account Activity API Ingestion:** Deploy an asynchronous webhook listener subscribing to real-time Twitter/X Account Activity API streams. Incoming @AmazonHelp customer mentions and direct messages (DMs) are ingested into an event queue.
- **Sub-Second Processing Loop:** Messages undergo automated normalization, language verification, and pipeline evaluation (intent classification, vector precedent retrieval, and grounding validation) within sub-second execution windows.
- **Bifurcated Dispatch Engine:**
- **Autonomous Dispatch (`OK_AUTO` , 17% of volume):** Replies meeting all six safety criteria are immediately posted via the Twitter REST v2 API with zero human friction.
- **Escalation Triage (`ESCALATE` , 83% of volume):** Messages failing any gate condition trigger an automated webhook payload to enterprise agent queues (Zendesk / Slack triage channels). The dispatch bundle includes the customer text, pre-drafted response, cited historical precedent IDs, and explicit failure reason code (e.g., `HIGH_RISK` , `WEAK_EVIDENCE`), enabling 1-click human agent review and dispatch.

Track B: Model Context Protocol (MCP) Server Tooling (`support-agent-mcp`)

- **Standardized Agent Interoperability:** Implement an enterprise MCP server exposing the support agent's capabilities over standardized JSON-RPC protocols (stdio and SSE transports).
- **Modular Tool Suite:** External AI reasoning systems—including Claude Desktop, Cursor AI, LangChain/LlamaIndex pipelines, and autonomous multi-agent swarms—can interact with customer support workflows via four atomic, sandboxed tools:
 1. `classify_intent(text: str)` : Normalizes messy customer queries into structured operational intents with calibrated confidence.
 2. `retrieve_precedents(query: str, top_k: int = 3)` : Performs semantic cosine search over the 3,334 verified, deflection-free historical brand resolution pairs.
 3. `draft_grounded_reply(query: str, precedent_ids: list[str])` : Synthesizes an empathetic, character-constrained resolution response conditioned strictly on retrieved evidence.
 4. `evaluate_policy_gate(draft: str, intent: str, precedents: list)` : Executes the full 6-point deterministic safety audit, verifying citations, scanning risk keywords, and returning an enforceable `OK_AUTO` or `ESCALATE` mandate.
- **Safe Agent Orchestration:** This decoupling enables external AI agents to leverage verified enterprise support domain knowledge while guaranteeing strict adherence to deterministic safety policies.

Track C: Algorithmic Refinements & Decision Log

- **Inter-Rater Agreement (κ):** Double-blind adjudication of 30 flagged boundary cases with a second domain annotator to calculate Cohen's κ and expand the golden benchmark to 250 verified items.
- **Three-State "Failed Attempt" Intent:** Decouple routine transit delays from lost shipments to eliminate the top source of classification confusion (32% of errors).
- **NLI-Based Grounding Validator:** Replace strict regex checks with a local natural language inference (NLI) model to verify premise-hypothesis entailment for complex policy claims without false regex trips.

8. Architectural Decision Log (15 Core Decisions)

Documented in `DECISIONS.md`: (1) Brand selected by groundable volume, deflections filtered; (2) Evaluation metrics hand-implemented in NumPy; (3) Coverage-vs-unsafe curve as headline result; (4) Escalation tuned for recall on must-escalate; (5) Human evaluation primary, LLM judge secondary cross-check; (6) Local embeddings and embedded Qdrant for \$0 replay; (7) Synthetic fixture and direct SDK invocations; (8) Cache miss in replay treated as hard error; (9) Pluggable embeddings with NumPy SVD fallback; (10) Operational intent band $k \in [4, 9]$; (11) Frozen 5+other taxonomy; (12) Stratified golden benchmark locked across 4 passes; (13) NaN-hardened detectors and focused escalation keywords; (14) Intent-evidence agreement ($\geq 2/3$) gating; (15) Quota and model lifecycle managed via deterministic replay caching.